

UNO Java Swing Project – Product Backlog

Project: UNO Card Game
Platform: Java + Swing

1. Project Idea

Java Gui project to implement a playable version of the UNO card game. The game will be designed for one human player against a computer opponent. The project will focus on object-oriented programming, modular design, and separation of game logic from the user interface.

2. Planned Advanced Technical Topics

Advanced Topic 1 – Game-State Persistence, Serialization, and Compression

- **Encapsulation:**
- **Decapsulation:**
- **Serialization:**
- **Deserialization:**
- **Compression:**
- **Decompression:**

Advanced Topic 2 – Computer Bot Decision-Making

- **Basic Bot Logic:** Implement a simple bot that plays legal cards based on straightforward rules.
 - **Result-Based Bot Logic:** Enhance the bot to evaluate possible moves and choose the best one based on the current game state, potentially using a scoring system or predefined rules.
 - **Advanced Bot Logic (Stretch Goal):** Implement a more sophisticated bot using algorithms like Minimax or a neural network
-

3. Product Backlog

#	Priority	Name	How to Demo	Notes
1	Must	Create project structure	Open the project and show the main packages/ classes compiling successfully.	Create a clear structure for model, game logic, bot logic, persistence, and GUI. Possible classes: <code>Card</code> , <code>Deck</code> , <code>Player</code> , <code>Game</code> , <code>Bot</code> , <code>GameFrame</code> .
2	Must	Create UNO Card model	Create several cards in code and print/show their colour, value, and type correctly.	Support Red, Yellow, Green, Blue and Wild cards. Include number cards and action cards.

3	Must	Create and shuffle UNO deck	Start the program and demonstrate that a full deck is created and shuffled into a different order.	The deck should contain the card types needed for the selected UNO rules.
4	Must	Deal starting hands	Start a new game and show that the player and computer each receive the correct number of cards.	Remaining cards become the draw pile. One card starts the discard pile.
5	Must	Implement basic card matching rules	Attempt to play valid and invalid cards and show that only legal cards are accepted.	A card can normally be played when colour, number, or action type matches the top discard card. Wild cards are exceptions.
6	Must	Implement player turns	Play several turns and show that control alternates correctly between player and computer.	Game state should clearly track the current player.

7	Must	Draw card action	Press the Draw button when no suitable card is available and show a card being added to the player's hand.	The game should continue according to the rules chosen by the team.
8	Must	Build main Swing game window	Run the application and show the player's cards, top discard card, draw pile/button, and turn information.	Use a simple functional layout first. Visual polish can come later.
9	Must	Make cards clickable	Click a card in the player's hand and show that a legal card is played while an illegal card is rejected.	Buttons or custom card components can be used.
10	Must	Implement Skip card	Play a Skip card and show that the opponent loses the appropriate turn.	Must work for both the human and computer player.
11	Must	Implement Reverse card	Play a Reverse card and show the expected behaviour.	With two players, Reverse can behave like Skip. This rule should be documented.

12	Must	Implement Draw Two card	Play a Draw Two and show the opponent receiving two cards and the turn continuing correctly.	Avoid stacking rules unless specifically added later.
13	Must	Implement Wild card colour selection	Play a Wild card and choose the next active colour using a Swing dialog or colour buttons.	Computer should also be able to choose a colour automatically.
14	Must	Implement Wild Draw Four	Play a Wild Draw Four and show colour selection plus four cards being added to the opponent's hand.	We can use a simplified rule unless there is enough time to implement the official challenge rule.
15	Must	Add basic computer opponent	Play several rounds and show the computer choosing and playing legal cards without user input.	First version may simply choose from available legal moves using straightforward rules.

16	Must	Detect UNO and game winner	Reduce a hand to one card and then zero cards and show the correct UNO/ win messages.	A visible UNO button or automatic UNO indication can be used.
17	Should	Add New Game / Restart	Finish or interrupt a game, press New Game, and show a fresh shuffled game starting correctly.	Reset all game state without restarting the Java application.
18	Should	Implement result-based bot logic	Play several turns and demonstrate that the bot evaluates possible moves instead of always choosing the first legal card.	The bot can look through a rule/ result file or predefined scoring data and select its next move based on the current situation. This is part of Advanced Topic 2.
19	Could	Add simple card and turn animations	Play/draw a card and show a small visual transition, highlight, or timed message.	Keep animations lightweight. They are visual polish, not a core technical requirement.

20	Should	Encapsulate game state for saving	Start a game and show that all important state can be collected into a dedicated save-state object or structure.	Part of Advanced Topic 1. Keep persistence logic separate from normal gameplay where possible.
21	Should	Serialize and save game state	Start a game, play several turns, save it, and show that the game state is written to a file.	Research Java serialization or another structured serialization approach.
22	Should	Compress saved game data	Save a game and demonstrate that the serialized data is compressed before storage.	Research suitable Java compression streams or libraries available within the project rules.
23	Should	Decompress and deserialize saved game	Load a saved file and demonstrate that hands, piles, active colour, and turn are restored correctly.	Include decompression, deserialization, and basic handling for invalid or missing files.

24	Should	Add simple score system	Complete a game and show the winner receiving points or a win counter.	Keep scoring simple if official UNO scoring would add unnecessary complexity.
25	Should	Add start screen and instructions	Launch the program and show a simple menu with Start Game, Load Game, Instructions, and Exit.	Instructions should explain the rules used in this version.
26	Should	Improve GUI styling	Show a clean game screen with readable cards, colours, buttons, spacing, and status messages.	Focus on usability rather than complex graphics or heavy animations.
27	Must	Test core game rules	Run automated or repeatable tests for deck creation, matching rules, drawing, action cards, bot decisions, and win detection.	Use JUnit if available/allowed, plus manual GUI testing where appropriate.

28	Must	Handle edge cases	Demonstrate that the game does not crash when the draw pile becomes empty, a save file is missing, or unusual action-card sequences occur.	Rebuild the draw pile from the discard pile when necessary.
29	Could	Add selectable bot difficulty	Start a game and choose between different levels of computer behaviour.	Example: Easy chooses simple valid moves, Normal uses the result-based decision system.
30	Must	Write README.md	Open the README and follow the instructions to build and run the game successfully.	Include Java version, how to build/run the project, controls, UNO rules used, bot behaviour, save/load information, and known limitations.

31	Could / Should	Create an advanced bot using Minimax or a neural network	Run a match and compare the advanced bot's decisions with the normal result-based bot.	Preferred option is a well-designed Minimax/search-based approach. A small neural-network experiment may be attempted if realistic and permitted. This is a stretch extension of Advanced Topic 2, not required for the basic game to work.
----	----------------	--	--	---

4. Definition of Done

A backlog item is considered complete when:

1. The feature is implemented.
2. The project compiles and runs.
3. All tests pass.
4. The feature does not break previously completed core functionality.
5. Important code is reasonably organised